

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное обеспечение

Дисциплина «Алгоритмы и структуры данных»

Отчёт по блоку задач №1

Выполнил
Линейский Аким Евгеньевич
Р3215
Преподаватель
Тараканов Денис Сергеевич

Санкт-Петербург, 2026

1. Задача А. Агроном-любитель

1.1. Условие задачи

А. Агроном-любитель

✓ Полное решение

Ограничение времени	2 секунды
Ограничение памяти	64 Мб
Ввод	стандартный ввод или agro.in
Вывод	стандартный вывод или agro.out

Городской школьник Лёша поехал на лето в деревню и занялся выращиванием цветов. Он посадил n цветков вдоль одной длинной прямой грядки, и они успешно выросли. Лёша посадил множество различных видов цветков, i -й от начала грядки цветок имеет вид a_i , где a_i — целое число, номер соответствующего вида в «Каталоге юного агронома».

Теперь Лёша хочет сделать фотографию выращенных им цветов и выложить ее в раздел «мои грядки» в социальной сети для агрономов «ВКомпосте». На фотографии будет виден отрезок из одного или нескольких высаженных подряд цветков.

Однако он заметил, что фотография смотрится не очень интересно, если на ней много одинаковых цветков подряд. Лёша решил, что если на фотографии будут видны три цветка одного вида, высаженные подряд, то его друзья — специалисты по эстетике цветочных фотографий — поставят мало лайков.

Помогите ему выбрать для фотографирования как можно более длинный участок своей грядки, на котором нет трех цветков одного вида подряд.

Формат ввода

В первой строке содержится целое число n ($1 \leq n \leq 200\,000$) — количество цветов на грядке.

Во второй строке содержится n целых чисел a_i ($1 \leq a_i \leq 10^9$), обозначающих вид очередного цветка. Одинаковые цветки обозначаются одинаковыми числами, разные — разными.

Формат вывода

Выведите номер первого и последнего цветка на самом длинном искомом участке. Цветки нумеруются от 1 до n .

Если самых длинных участков несколько, выведите участок, который начинается раньше.

Пример

Ввод	Вывод
6 5 6 6 6 23 9	3 6

1.2. Идея решения

Для решения задачи используется метод двух указателей (скользящее окно). Мы поддерживаем левую границу текущего корректного участка l (изначально 0) и итерируем правую границу r от 2 до $n - 1$.

Если в процессе расширения окна обнаруживается, что текущий цветок совпадает с двумя предыдущими ($a[r] == a[r - 1] \ \&\& \ a[r] == a[r - 2]$), это означает нарушение

условия. Чтобы исправить нарушение и сохранить максимальную длину, левую границу l необходимо сдвинуть сразу на позицию $r - 1$ (оставляя два одинаковых цветка, но не три подряд).

На каждом шаге мы вычисляем текущую длину участка $cur_len = r - l + 1$ и, если она строго больше max_len , обновляем максимальную длину и сохраняем индексы начала и конца. Строгое неравенство автоматически гарантирует выбор участка, который начался раньше, в случае равенства длин.

1.3. Исходный код программы

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main() {
7     int n;
8     cin >> n;
9
10    vector<int> a(n);
11    for (int i = 0; i < n; i++) {
12        cin >> a[i];
13    }
14
15    if (n <= 2) {
16        cout << 1 << " " << n << endl;
17        return 0;
18    }
19
20    int l = 0;
21    int max_len = 2;
22    int bst_l = 0, bst_r = 1;
23
24    for (int r = 2; r < n; r++) {
25        if (a[r] == a[r - 1] && a[r] == a[r - 2]) {
26            l = r - 1;
27        }
28
29        int cur_len = r - l + 1;
30        if (cur_len > max_len) {
31            max_len = cur_len;
32            bst_l = l;
33            bst_r = r;
34        }
35    }
36    cout << bst_l + 1 << " " << bst_r + 1 << endl;
37    return 0;
38 }
```

2. Задача В. Зоопарк Глеба

2.1. Условие задачи

В. Зоопарк Глеба

✓ Полное решение

Ограничение времени	1 секунда
Ограничение памяти	256 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Недавно Глеб открыл зоопарк. Он решил построить его в форме круга и, естественно, обнёс забором. Глеб взял вас туда начальником охраны. Казалось бы все началось так хорошо, но именно в вашу первую смену все животные разбежались. В зоопарке n животных различных видов, также под каждый из видов есть свои ловушки. К сожалению некоторые животные враждуют с друг другом в природе (они обозначены разными буквами), а зоопарк обнесён забором и имеет форму круга. С помощью камер, удалось выяснить, где находятся все животные. Умная система поддержки жизнедеятельности зоопарка уже просканировала зоопарк и вывела id всех животных и ловушек в том порядке, в котором они видны из центра зоопарка. Получилось так, что все животные и все ловушки находятся на краю зоопарка. Вы хотите понять, могут ли животные прийти в свою ловушку так, чтобы их путь не пересекался с другими. Если да, также предъявите какую-нибудь из схем поимки животных.

Формат ввода

На вход подается строка из $2 \cdot n$ символов латинского алфавита, где маленькая буква - животное, а большая - ловушка. Размер строки не более 100000.

Формат вывода

Требуется вывести "Impossible", если решения не существует или "Possible", если можно вернуть всех животных в клетки. В случае если можно, то для каждой ловушки в порядке обхода требуется вывести индекс животного в ней.

Пример 1

Ввод 	Вывод 
ABba	Possible 2 1

Пример 2

Ввод 	Вывод 
ABab	Impossible

2.2. Идея решения

Так как объекты расположены по кругу, а пути не должны пересекаться, задача эквивалентна проверке правильности скобочной последовательности, где открывающими и закрывающими «скобками» являются соответствующие строчные (животные) и заглавные (ловушки) буквы одного типа.

Мы используем структуру данных **стек** для симуляции процесса. Обходим строку слева направо. Каждому животному и каждой ловушке присваивается свой порядковый индекс от 1 до n по мере их появления. При обработке текущего символа мы проверяем, совпадает ли он по типу (но отличается регистром) с элементом на вершине стека. Если они образуют пару «животное-ловушка» одного вида, мы связываем их индексы в массиве `trap2animal` и удаляем элемент из стека. В противном случае текущий символ добавляется в стек.

Если после полного обхода строки стек оказался пуст, значит, все животные успешно и бесконфликтно распределились по ловушкам. Выводится **Possible** и схема связей. Если стек не пуст — выводится **Impossible**.

2.3. Исходный код программы

```
1 #include <cctype>
2 #include <cstddef>
3 #include <iostream>
4 #include <stack>
5 #include <string>
6 #include <vector>
7 using namespace std;
8
9 struct Element {
10     char val;
11     int id;
12 };
13
14 bool is_match(char a, char b) {
15     if (tolower(a) != tolower(b))
16         return false;
17     return (islower(a) && isupper(b)) || (isupper(a) && islower(b));
18 }
19
20 int main() {
21     string s;
22     if (!(cin >> s))
23         return 0;
24
25     int n = s.length() / 2;
26     vector<int> trap2animal(n + 1);
27     stack<Element> st;
28
29     int animal_idx = 0;
30     int trap_idx = 0;
31
32     for (size_t i = 0; i < s.length(); ++i) {
33         char curChar = s[i];
34         int cur_id;
35
36         if (islower(curChar)) {
37             cur_id = ++animal_idx;
38         } else {
39             cur_id = ++trap_idx;
40         }
41
42         if (!st.empty() && is_match(st.top().val, curChar)) {
43             if (isupper(curChar)) {
44                 trap2animal[cur_id] = st.top().id;
45             } else {
```

```

46     trap2animal[st.top().id] = cur_id;
47     }
48     st.pop();
49 } else {
50     st.push({curChar, cur_id});
51 }
52 }
53
54 if (st.empty()) {
55     cout << "Possible" << endl;
56     for (int i = 1; i <= n; ++i) {
57         cout << trap2animal[i] << (i == n ? "" : " ");
58     }
59     cout << endl;
60 } else {
61     cout << "Impossible" << endl;
62 }
63
64 return 0;
65 }

```

3. Задача С. Конфигурационный файл

3.1. Условие задачи

С. Конфигурационный файл

✓ Полное решение

Ограничение времени	1 секунда
Ограничение памяти	64 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Вадим разрабатывает парсер конфигурационных файлов для своего проекта. Файл состоит из блоков, которые выделяются с помощью символов «{» — начало блока, и «}» — конец блока. Блоки могут вкладываться друг в друга. В один блок может быть вложено несколько других блоков.

В конфигурационном файле встречаются переменные. Каждая переменная имеет имя, которое состоит из не более чем десяти строчных букв латинского алфавита. Переменным можно присваивать числовые значения. Изначально все переменные имеют значение 0.

Присваивание нового значения записывается как `<variable>=<number>`, где `<variable>` — имя переменной, а `<number>` — целое число, по модулю не превосходящее 10^9 . Парсер читает конфигурационный файл построчно. Как только он встречает выражение присваивания, он присваивает новое значение переменной. Это значение сохраняется до конца текущего блока, а затем восстанавливается старое значение переменной. Если в блок вложены другие блоки, то внутри тех из них, которые идут после присваивания, значение переменной также будет новым.

Кроме того, в конфигурационном файле можно присваивать переменной значение другой переменной. Это действие записывается как `<variable1>=<variable2>`. Прочитав такую строку, парсер присваивает текущее значение переменной `variable2` переменной `variable1`. Как и в случае присваивания константного значения, новое значение сохраняется только до конца текущего блока. После окончания блока переменной возвращается значение, которое было перед началом блока.

Для отладки Вадим хочет напечатать присваиваемое значение для каждой строки вида `<variable1>=<variable2>`. Помогите ему отладить парсер.

Формат ввода

Входные данные содержат хотя бы одну и не более 10^5 строк. Каждая строка имеет один из четырех типов:

- { — начало блока;
- } — конец блока;
- `<variable>=<number>` — присваивание переменной значения, заданного числом;
- `<variable1>=<variable2>` — присваивание одной переменной значения другой переменной. Переменные `<variable1>` и `<variable2>` могут совпадать.

Гарантируется, что ввод является корректным и соответствует описанию из условия. Ввод не содержит пробелов.

Формат вывода

Для каждой строки типа `<variable1>=<variable2>` выведите значение, которое было присвоено.

3.2. Идея решения

Для корректной реализации областей видимости (лексических контекстов) используются хэш-таблица и стек:

1. Каждая переменная имеет свой собственный стек значений в структуре `unordered_map<string, stack<int>> vals`. Это позволяет легко получать текущее актуальное значение переменной через метод `top()` и мгновенно возвращаться к предыдущему значению через `pop()`.
2. Также поддерживается стек блоков `vector<vector<string>> scopes`, где для каждого текущего уровня вложенности хранится список имен переменных, которые были изменены или созданы в текущем блоке.

При встрече открывающей фигурной скобки `{` мы создаем новый пустой список переменных для текущего вложенного блока. При встрече закрывающей скобки `}` мы обходим список переменных, измененных в закрывающемся блоке, и удаляем верхнее значение из стека каждой такой переменной (если стек переменной опустел полностью, удаляем её из хэш-таблицы). Затем удаляем сам завершённый блок из `scopes`.

При обработке строки присваивания `var1=r` сначала вычисляется значение правой части `r`. Если это число, парсим его; если переменная — берем значение с вершины её стека. Если выполнялось присваивание переменной другой переменной, это вычисленное значение выводится в стандартный поток вывода. В конце значение помещается в стек для `var1`, а её имя записывается в текущую область видимости для отслеживания деструкции блока.

3.3. Исходный код программы

```
1 #include <iostream>
2 #include <stack>
3 #include <string>
4 #include <unordered_map>
5 #include <vector>
6 using namespace std;
7
8 int main() {
9     unordered_map<string, stack<int>>> vals;
10    vector<vector<string>>> scopes;
11    scopes.push_back({});
12
13    string line;
14    while (cin >> line) {
15        if (line == "{") {
16            scopes.push_back({});
17        } else if (line == "}") {
18            for (string var_name : scopes.back()) {
19                vals[var_name].pop();
20
21                if (vals[var_name].empty()) {
22                    vals.erase(var_name);
23                }
24            }
25            scopes.pop_back();
26
27        } else {
28            size_t pos = line.find('=');
29            string var1 = line.substr(0, pos);
30            string r = line.substr(pos + 1);
31
32            int val;
33            if (isdigit(r[0]) || (r.length() > 1 && r[0] == '-' && isdigit(r[1]))) {
34                val = stoi(r);
```



```
35     } else {
36         val = vals.count(r) ? vals[r].top() : 0;
37         cout << val << "\n";
38     }
39     scopes.back().push_back(var1);
40     vals[var1].push(val);
41 }
42 }
43 return 0;
44 }
```

4. Задача D. Профессор Хаос

4.1. Условие задачи

D. Профессор Хаос

✓ Полное решение

Ограничение времени	1 секунда
Ограничение памяти	64 Мб
Ввод	стандартный ввод или chaos.in
Вывод	стандартный вывод или chaos.out

В секретной лаборатории профессора Хаоса проходит эксперимент по выращиванию особо опасных бактерий. В начале первого дня эксперимента у Хаоса имеется a особо опасных бактерий.

Каждый день эксперимента устроен следующим образом. Рано утром профессор достает из контейнера все свои бактерии и помещает их в инкубатор, где бактерии начинают делиться. Вместо каждой бактерии образуется b новых бактерий.

После извлечения бактерий из инкубатора c из них используются для проведения различных опытов и затем уничтожаются. Если после извлечения из инкубатора имеется менее c бактерий, для проведения опытов используются все имеющиеся бактерии, и эксперимент заканчивается.

Оставшиеся бактерии в конце дня необходимо поместить в контейнер и продолжить использовать в эксперименте. Однако в контейнер можно поместить не более d бактерий, поэтому если число оставшихся бактерий больше d , то в контейнер помещаются d бактерий, а остальные уничтожаются.

Теперь профессор Хаос хочет выяснить, сколько особо опасных бактерий будет у него в контейнере после k -го дня эксперимента. Помогите ему найти ответ на этот вопрос.

Формат ввода

В единственной строке входного файла содержится пять целых чисел a, b, c, d и k ($1 \leq a, b \leq 1000, 0 \leq c \leq 1000, 1 \leq d \leq 1000, a \leq d, 1 \leq k \leq 10^{18}$).

Формат вывода

Выведите одно число — количество бактерий у Хаоса к концу k -го дня. Если эксперимент завершится в k -й день или ранее, выведите число 0.

4.2. Идея решения

Основная сложность задачи заключается в том, что число дней k может быть крайне большим, что полностью исключает возможность прямого моделирования процесса за линейное время. Однако, поскольку максимальная вместимость контейнера $d \leq 1000$, количество возможных *состояний* системы (число живых бактерий в конце любого дня) жестко ограничено — от 0 до d .

Это свойство гарантирует, что динамика популяции бактерий очень быстро либо зациклится, либо стабилизируется. Мы можем проводить пошаговое моделирование, отслеживая два терминальных условия для досрочного выхода из цикла:

1. Если после размножения количество бактерий становится меньше или равно числу бактерий, забираемых на опыт, популяция полностью вымирает, и ответ — 0.
2. Если в конце текущего дня число бактерий `cur_a` в точности равно числу бактерий в

конце предыдущего дня `prev_a`, система перешла в стабильное стационарное состояние. Дальнейшие итерации не изменяют значение, поэтому можно досрочно выполнить `break`.

Так как различных состояний не больше 1001, процесс стабилизируется или завершится не более чем за 1001 шаг. Использование типа данных `long long` предотвращает переполнение при промежуточном вычислении `cur_a * b`.

4.3. Исходный код программы

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     long long a, b, c, d, k;
6     cin >> a >> b >> c >> d >> k;
7
8     long long cur_a = a;
9     for (long long day = 1; day <= k; ++day) {
10         long long prev_a = cur_a;
11         cur_a = cur_a * b;
12         if (cur_a <= c) {
13             cur_a = 0;
14             break;
15         }
16         cur_a -= c;
17         if (cur_a > d) {
18             cur_a = d;
19         }
20         if (cur_a == prev_a) {
21             break;
22         }
23     }
24     cout << cur_a << endl;
25     return 0;
26 }
```